```
"""
Personal Expense Tracker with CSV
A beginner-friendly CLI program that:
  - Records daily expenses by
category
  - Saves them persistently to a CSV
file
  - Shows all expenses, totals by
category, and monthly totals
  - Validates all user input
"""

import csv
import os
from datetime import datetime

FILENAME = "expenses.csv"
FIELDS = ["date", "category",
"description", "amount"]
CATEGORIES = ["Food", "Transport",
"Shopping", "Bills", "Entertainment",
"Health", "Other"]


# ---------- File handling ----------
def ensure_file():
```

```
    """Create the CSV file with a
header row if it doesn't exist
yet."""
    if not os.path.exists(FILENAME):
        with open(FILENAME, "w",
newline="", encoding="utf-8") as
file:
            writer =
csv.DictWriter(file,
fieldnames=FIELDS)
            writer.writeheader()


def load_expenses():
    """Read all expenses from the CSV
file into a list of dictionaries."""
    ensure_file()
    expenses = []
    with open(FILENAME, "r",
newline="", encoding="utf-8") as
file:
        reader = csv.DictReader(file)
        for row in reader:
            try:
                row["amount"] =
float(row["amount"])
                expenses.append(row)
            except (ValueError,
KeyError, TypeError):
```

```
                continue  # skip
damaged rows
    return expenses


def save_expenses(expenses):
    """Rewrite the CSV file with the
given list of expenses."""
    with open(FILENAME, "w",
newline="", encoding="utf-8") as
file:
        writer = csv.DictWriter(file,
fieldnames=FIELDS)
        writer.writeheader()
        for exp in expenses:
            writer.writerow(exp)


def append_expense(expense):
    """Add one expense to the end of
the CSV file."""
    ensure_file()
    with open(FILENAME, "a",
newline="", encoding="utf-8") as
file:
        writer = csv.DictWriter(file,
fieldnames=FIELDS)
        writer.writerow(expense)
```

```
# ---------- Input helpers ----------
def get_choice(prompt,
valid_choices):
    while True:
        choice =
input(prompt).strip()
        if choice in valid_choices:
            return choice
        print(f"Invalid choice.
Please pick one of: {',
'.join(valid_choices)}")


def get_amount(prompt):
    """Keep asking until the user
enters a positive number."""
    while True:
        value = input(prompt).strip()
        try:
            amount = float(value)
        except ValueError:
            print("Invalid amount.
Please enter a number.")
            continue
        if amount <= 0:
            print("Amount must be
greater than zero.")
        else:
```

```
            return round(amount, 2)


def get_date(prompt):
    """Keep asking until the user
enters a valid date (YYYY-MM-DD) or
presses Enter for today."""
    while True:
        value = input(prompt).strip()
        if value == "":
            return
datetime.now().strftime("%Y-%m-%d")
        try:
            datetime.strptime(value,
"%Y-%m-%d")
            return value
        except ValueError:
            print("Invalid date. Use
the format YYYY-MM-DD (example: 2026-
10-18).")


def get_category():
    print("\nCategories:")
    for i, name in
enumerate(CATEGORIES, start=1):
        print(f"  {i}. {name}")
    choice = get_choice(f"Choose a
category (1-{len(CATEGORIES)}): ",
```

```
                          [str(i) for i
in range(1, len(CATEGORIES) + 1)])
    return CATEGORIES[int(choice) -
1]


# ---------- Features ----------
def add_expense():
    print("\n--- Add Expense ---")
    date = get_date("Date (YYYY-MM-
DD, press Enter for today): ")
    category = get_category()
    description = input("Description:
").strip() or "No description"
    amount = get_amount("Amount: ")

    append_expense({
        "date": date,
        "category": category,
        "description": description,
        "amount": amount,
    })
    print(f"Saved: {date} |
{category} | {description} |
{amount:.2f}")


def view_expenses():
    print("\n--- All Expenses ---")
```

```
    expenses = load_expenses()
    if not expenses:
        print("No expenses recorded
yet.")
        return

    print(f"{'No.':<5}{'Date':<12}
{'Category':<15}{'Description':<25}
{'Amount':>10}")
    print("-" * 67)
    for i, exp in enumerate(expenses,
start=1):
        print(f"{i:<5}{exp['date']:
<12}{exp['category']:<15}"
              f"{exp['description']
[:23]:<25}{exp['amount']:>10.2f}")
    print("-" * 67)
    print(f"{'Total':<57}
{sum(e['amount'] for e in
expenses):>10.2f}")


def category_summary():
    print("\n--- Summary by Category
---")
    expenses = load_expenses()
    if not expenses:
        print("No expenses recorded
yet.")
```

```
        return

    totals = {}
    for exp in expenses:
        totals[exp["category"]] =
totals.get(exp["category"], 0) +
exp["amount"]

    grand_total =
sum(totals.values())
    for category, total in
sorted(totals.items(), key=lambda
item: item[1], reverse=True):
        percent = (total /
grand_total) * 100
        print(f"{category:<15}
{total:>10.2f}   ({percent:.1f}%)")
    print("-" * 36)
    print(f"{'Total':<15}
{grand_total:>10.2f}")


def monthly_summary():
    print("\n--- Monthly Summary ---
")
    expenses = load_expenses()
    if not expenses:
        print("No expenses recorded
yet.")
```

```
        return

    months = {}
    for exp in expenses:
        month = exp["date"][:7]  #
"YYYY-MM"
        months[month] =
months.get(month, 0) + exp["amount"]

    for month in sorted(months):
        print(f"{month}:
{months[month]:.2f}")


def delete_expense():
    print("\n--- Delete Expense ---")
    expenses = load_expenses()
    if not expenses:
        print("No expenses to
delete.")
        return

    view_expenses()
    number = input("\nEnter the
number to delete (or press Enter to
cancel): ").strip()
    if number == "":
        print("Cancelled.")
        return
```

```
    if not number.isdigit() or not (1
<= int(number) <= len(expenses)):
        print("Invalid number.
Nothing was deleted.")
        return

    removed =
expenses.pop(int(number) - 1)
    save_expenses(expenses)
    print(f"Deleted:
{removed['date']} |
{removed['category']} |
{removed['amount']:.2f}")


# ---------- Main menu ----------
def main():
    ensure_file()
    print("=== Personal Expense
Tracker ===")
    while True:
        print("\nMain Menu")
        print("1. Add expense")
        print("2. View all expenses")
        print("3. Summary by
category")
        print("4. Monthly summary")
        print("5. Delete an expense")
        print("6. Exit")
```

```
        choice = get_choice("Choose
an option (1-6): ", ["1", "2", "3",
"4", "5", "6"])

        if choice == "1":
            add_expense()
        elif choice == "2":
            view_expenses()
        elif choice == "3":
            category_summary()
        elif choice == "4":
            monthly_summary()
        elif choice == "5":
            delete_expense()
        else:
            print("Your expenses are
saved in", FILENAME, "- Goodbye!")
            break


if __name__ == "__main__":
    main()
```